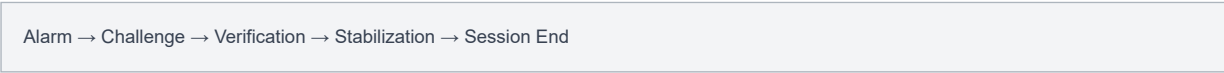


SMART WAKE & ROOM ACTIVITY SYSTEM (ESP32-Based Behavioral Alarm System)

PROJECT OVERVIEW

This is a **state-machine based intelligent alarm system** designed to ensure real wakefulness, not just turning off an alarm.

Instead of a simple beep-based alarm, the system guides the user through a **multi-stage wake transition process**:



Each stage ensures the user is not only awake temporarily, but actually active and stable.

CORE INNOVATION

Key Idea:

“Wake-up is not a single event, it is a controlled transition.”

The system prevents:

- turning off alarm and sleeping again
- fake wake-ups
- passive alarm dismissal

SYSTEM ARCHITECTURE

1. ALARM PHASE

- Triggered manually or scheduled
- Buzzer activates
- PIR monitoring begins

2. CHALLENGE PHASE (Cognitive activation layer)

Purpose:

Force interaction immediately after wake-up

Features:

- Random LED pattern (6-digit sequence concept)
- Button-based response system
- Reset button to replay pattern
- Must complete within time limit

Outcome:

- Success → Verification phase
- Timeout → alarm re-triggers

3. VERIFICATION PHASE

Purpose:

Confirm real wakefulness using behavior

Inputs:

- PIR motion (primary)

- Microphone activity (secondary support)

Logic:

- Must show distributed activity over time
- No long inactivity (>60 sec)
- 3-minute evaluation window

Outcome:

- PASS → Stabilization
- FAIL → Back to Challenge / Alarm

4. STABILIZATION PHASE

Purpose:

Prevent “sleep again after waking”

Duration:

- 5–10 minutes monitoring

Logic:

- Detect relapse into inactivity
- Confirm consistent wake behavior

Outcome:

- PASS → Session ends
- FAIL → Return to Challenge/Alarm



DATA LOGGING (SD CARD BASED)

Each full cycle = **ONE session row**

SessionID, StartTime, ChallengeTime,
VerificationResult, StabilizationStatus,
ResetCount, FinalOutcome

Storage strategy:

- Only write AFTER session ends
- Prevents corruption
- Supports long-term analytics



ANALYTICS APPROACH

Data is later analyzed using laptop tools (Streamlit / Python):

Metrics derived:

- Time to leave bed
- Wake stability duration
- Challenge completion speed

- Daily/weekly consistency patterns

SYSTEM DESIGN (IMPORTANT ENGINEERING POINTS)

State Machine Architecture:

ALARM → CHALLENGE → VERIFICATION → STABILIZATION → END

Timing Engine:

- millis-based non-blocking FSM
- no delay() usage
- independent timers per state

FAILURE HANDLING (VERY IMPORTANT FEATURE)

System is fault-tolerant:

Handles:

- Power cuts → state recovery via NVS
- PIR sensor stuck → fallback filtering
- system freeze → watchdog reset
- partial session crash → safe reset or recovery
- environmental false triggers → debounce + pattern validation

Rule:

System always returns to a safe ALARM state

POWER + USAGE DESIGN

- System always ON in idle mode
- Activates only on alarm event
- No unnecessary computation
- SD card acts as persistent memory
- Optional WiFi only for future dashboard mode

KEY TECHNICAL CONCEPTS USED

- ESP32 FSM (Finite State Machine)
- Event-driven architecture
- PIR motion sensing
- Microphone threshold detection (support layer)
- SD card logging (append-only session model)
- Non-blocking timer design
- NVS state recovery system

WHAT MAKES THIS PROJECT UNIQUE

✓ Not a simple alarm clock ✓ Behavioral wake validation system ✓ Multi-stage cognitive + physical engagement ✓ Crash-safe embedded system design ✓ Offline-first architecture (no dependency on WiFi) ✓ Data-driven future analytics ready

You said:

similar to previous one, give complete summary of the verification phase ?

ChatGPT said:

Got it 🧯 here's your **clean, implementation-ready Verification Phase summary (V1 LOCKED)** with flowchart and exact system behavior.

🧯 VERIFICATION PHASE — COMPLETE SYSTEM SUMMARY (V1 LOCKED)

🎯 PURPOSE

Verification Phase confirms:

“User is actually awake, not just completing a mechanical challenge.”

It validates **sustained wakefulness**, not instant response.

⚙️ WHEN VERIFICATION STARTS

Triggered only after Challenge Phase = PASS

System enters a **monitoring + validation window**.

🕒 DURATION

Fixed window = 3 minutes

Non-interruptible except failure or success conditions.

💡 CORE IDEA

Instead of asking the user to do something, system checks:

natural behavior consistency over time

📡 INPUT SOURCES

🟢 1. PIR Sensor (PRIMARY)

- Detects motion events
- Used for activity distribution analysis

🟡 2. Microphone (SECONDARY SUPPORT ONLY)

- Detects:
 - typing
 - object movement
 - environmental interaction noise

❌ Cannot decide PASS/FAIL alone

🔥 VERIFICATION LOGIC (V1 LOCKED RULES)

✓ RULE 1: ACTIVITY DISTRIBUTION

User must show:

multiple motion events spread across time window

Not a single burst.

✓ RULE 2: INACTIVITY LIMIT

No continuous inactivity > 60 seconds allowed

If exceeded → system assumes relapse into sleep.

✓ RULE 3: MINIMUM ACTIVITY REQUIREMENT

During 3 min window:

At least 2–3 distinct activity events required

(PIR or mic-supported)

✗ FAILURE CONDITIONS

Verification FAIL if ANY:

- No motion > 60 sec continuously
- No meaningful distributed activity
- Only single early burst then silence

→ Action:

- Send back to CHALLENGE or ALARM depending severity

✓ SUCCESS CONDITIONS

Verification PASS if:

- Activity is distributed over time
- No long inactivity gaps
- At least minimal PIR/mic activity exists

→ Action:

- Move to STABILIZATION phase

DATA LOGGED (SUMMARY ONLY)

VerificationStartTime
VerificationEndTime
MotionEventCount
MaxInactivityDuration
Result (PASS/FAIL)

SYSTEM BEHAVIOR RULES

- Non-blocking (millis-based FSM)
- PIR continuously monitored
- Mic only enhances confidence
- No external interaction required from user

KEY DESIGN INSIGHT

Verification Phase is NOT:

✗ “do task to prove awake”

It IS:

behavioral consistency detection over time window

FLOWCHART — VERIFICATION PHASE



FINAL LOCKED SUMMARY

VERIFICATION PHASE =

✓ 3-minute behavioral monitoring window ✓ PIR-based motion distribution check ✓ Mic as supporting signal only ✓ Inactivity threshold (60 sec) ✓ PASS → Stabilization ✓ FAIL → Challenge/Alarm

FINAL INSIGHT

This phase is your:

“truth detector between fake wake and real wake”

It ensures:

- user didn't just press buttons blindly
- user is actually active over time
- wake-up is sustained, not momentary

You said:

then,, followed by wake stabilization

ChatGPT said:

Perfect — let's lock the **Wake Stabilization Phase** cleanly so your whole system becomes fully complete and consistent 🔥

🔵 WAKE STABILIZATION PHASE — COMPLETE SUMMARY (V1 LOCKED)

🎯 PURPOSE

After verification PASS:

| Ensure the user does NOT fall back asleep immediately.

This phase confirms:

wake state is stable, not temporary

⚙️ WHEN IT STARTS

Triggered only after VERIFICATION = PASS

System transitions automatically.

🕒 DURATION

5 to 10 minutes (configurable window)

This is a passive monitoring phase.

💡 CORE IDEA

Unlike earlier phases:

- Challenge → forces action
- Verification → checks behavior
- Stabilization → checks **consistency over time**

No tasks, only observation

📡 INPUT SOURCES

🟢 PIR Sensor (PRIMARY)

- Detects motion frequency
- Detects inactivity duration

🟡 Microphone (OPTIONAL SUPPORT)

- Detects ambient activity (typing, movement noise)
- Used only for confirmation, not decision-making

🔥 STABILIZATION LOGIC (V1 LOCKED RULES)

✓ RULE 1: CONTINUOUS INACTIVITY LIMIT

If no motion for > 2–3 minutes → suspicious

Indicates possible sleep relapse.

✓ RULE 2: PERIODIC ACTIVITY EXPECTATION

During window:

User must show occasional movement bursts

(Not continuous movement, just natural wake behavior)

✓ RULE 3: RELAPSE DETECTION

If pattern is:

activity → long silence → no recovery movement

→ likely sleep again

✗ FAILURE CONDITIONS

Stabilization FAIL if:

- Long inactivity (>2–3 min)
- No meaningful motion across full window
- Sleep relapse pattern detected

→ Action:

Return → CHALLENGE or ALARM (soft restart)

✓ SUCCESS CONDITIONS

Stabilization PASS if:

- Small periodic motion exists
- No long inactivity gaps
- Stable wake behavior maintained

→ Action:

SESSION COMPLETE → LOG DATA → IDLE MODE

📄 DATA LOGGED (FINAL SESSION SUMMARY)

SessionID
StabilizationStartTime
StabilizationEndTime
InactivityMaxDuration
RelapseDetected (YES/NO)
FinalResult (SUCCESS/FAIL)

⚙️ SYSTEM BEHAVIOR RULES

- Fully non-blocking FSM
- No user interaction required
- Only observation mode
- Runs after verification automatically

💡 KEY DESIGN INSIGHT

Stabilization is NOT:

✗ another test ✗ another challenge ✗ another verification

It is:

wake state validation over time

FLOWCHART — WAKE STABILIZATION PHASE



FINAL LOCKED SUMMARY

WAKE STABILIZATION PHASE =

✓ Passive monitoring phase ✓ 5-10 min observation window ✓ Detects sleep relapse ✓ PIR-based inactivity tracking ✓ No user interaction required ✓ Final confirmation stage ✓ Leads to session completion

FINAL SYSTEM INSIGHT (FULL PROJECT VIEW)

Now your full system is:

ALARM → CHALLENGE → VERIFICATION → STABILIZATION → SESSION LOGGED

You said:

give the complete summary again for challenge phase

ChatGPT said:

Got it 🟡 here's your **clean, final, implementation-ready Challenge Phase summary (V1 LOCKED)** — structured so you can directly map it into ESP32 firmware.

CHALLENGE PHASE — COMPLETE SUMMARY (V1 LOCKED)

PURPOSE

Challenge Phase is the **first active wake-up barrier**.

It ensures:

User is fully responsive and cognitively engaged after alarm trigger

It breaks:

- automatic alarm dismissal
- half-sleep button pressing
- muscle-memory wake-off behavior

WHEN IT STARTS

Triggered immediately after ALARM phase
OR after system wake trigger (PIR or alarm event)

CORE IDEA

Instead of "press to stop alarm", system forces:

pattern recognition + active response + motor engagement

So user cannot turn off alarm blindly.

CHALLENGE STRUCTURE (V1)

1. PATTERN GENERATION (ESP32)

- ESP32 generates random sequence
- Sequence length fixed (example: 6 steps)

Example:

2 → 3 → 1 → 2 → 3 → 1

Mapping:

- 1 = Button A / LED1
- 2 = Button B / LED2
- 3 = Button C / LED3

2. LED DISPLAY SYSTEM

- Sequence is shown using LED blinking
- Each step shown sequentially
- User must memorize pattern

LED blink = hidden instruction system

No text display required.

3. USER INPUT SYSTEM

User responds using:

- 3 push buttons (or similar inputs)

Rules:

- Must enter same sequence
- Order matters strictly

4. RESET FUNCTION

- Reset button replays LED sequence
- Helps user re-learn pattern
- Does NOT reset timer

5. TIMER LIMIT

Max duration = 2 minutes

Non-blocking countdown.

FAILURE CONDITIONS

Challenge FAIL if ANY:

- Timeout exceeds 2 minutes
- Wrong sequence entered
- No meaningful interaction

→ Action:

Return to ALARM phase
Restart challenge cycle

SUCCESS CONDITION

Challenge PASS if:

Correct sequence entered within time limit

→ Action:

Move to VERIFICATION phase

DATA LOGGED (SUMMARY ONLY)

SessionID
ChallengeStartTime
ChallengeEndTime
AttemptsCount
ResetCount
Result (PASS/FAIL)

SYSTEM BEHAVIOR RULES

- Fully non-blocking (millis-based FSM)
- PIR monitoring continues in background
- No delay() usage

- LED + buzzer operate via state machine
- Buttons debounced (to avoid false triggers)

🧠 KEY DESIGN INSIGHT

Challenge Phase is NOT:

❌ simple button press ❌ alarm stop mechanism

It IS:

cognitive + motor + memory activation layer

📄 FLOWCHART — CHALLENGE PHASE



🔒 FINAL LOCKED SUMMARY

CHALLENGE PHASE =

✓ Random LED sequence generator ✓ 6-step memory pattern system ✓ 3-button input validation ✓ Reset replay feature ✓ 2-minute strict timer ✓ Failure → Alarm restart ✓ Success → Verification phase

🚀 FINAL INSIGHT

This phase acts as:

“wake-up cognitive activation gate”

- awake enough to think
- coordinated enough to respond
- not acting in sleep inertia